

PULSENEX

Website Availability Monitor for Linux

Project Report

Prepared by: Afsa Taj

Dubai, UAE

July 2026

Table of Contents

Table of Contents.....2

1. Executive Summary.....3

2. Project Objectives3

3. System Architecture.....3

4. Features Overview4

 4.1 Live Terminal Dashboard4

 4.2 Outage Detection.....4

 4.3 Latency Monitoring.....4

 4.4 SSL Certificate Expiry Tracking4

 4.5 Content / Keyword Validation4

 4.6 Persistent Local Storage & Reporting4

 4.7 Demo Mode4

5. Technology Stack5

6. Alerting Rules5

7. Installation & Deployment.....5

8. Screenshots & Demonstration.....6

9. Testing & Validation.....6

10. Future Enhancements.....9

11. Conclusion.....9

1. Executive Summary

PulseNex is a lightweight website and API availability monitor built for Linux, designed as a self-hosted alternative to paid uptime-monitoring SaaS products such as UptimeRobot or Pingdom for freelancers and small teams who want full control of their monitoring data. It runs as a single Python script with no external services, checking a configurable list of URLs on a recurring schedule, measuring HTTP response time and status codes, verifying SSL certificate expiry, and optionally validating that expected content is present on the page.

Results are shown on a live, auto-refreshing terminal dashboard and are simultaneously persisted to a local SQLite database, giving both real-time visibility and a historical record suitable for uptime reporting to clients or for personal infrastructure monitoring.

2. Project Objectives

- Continuously monitor a configurable list of websites/APIs for availability and performance.
- Detect and alert on site outages (connection failures, timeouts, 5xx errors) in real time.
- Detect slow response times against a configurable latency threshold.
- Track SSL/TLS certificate expiry and warn before certificates lapse.
- Optionally verify that expected page content (a keyword) is present, to catch silent failures such as broken deployments that still return HTTP 200.
- Persist all checks and alerts locally with zero external dependencies, for historical uptime reporting.
- Present a clean, professional live dashboard suitable for client demos and NOC-style screen displays.

3. System Architecture

PulseNex is built around a simple scheduler loop. Each monitored site has its own check interval; on every scheduler tick, PulseNex determines which sites are due for a check, performs an HTTP(S) request and (for HTTPS sites) an SSL certificate inspection, evaluates the result against the alerting rules, and updates both the live dashboard and the SQLite database.

Site Registry	Check Engine	Storage Layer	Dashboard
SQLite 'sites' table — URL, name, keyword, per-site check interval	HTTP(S) GET via requests + TLS handshake for cert expiry (ssl module)	SQLite database at ~/.pulsenex/pulsenex.db — sites, checks, alerts tables	Rich Live terminal UI — site status table, status summary, recent alerts feed

Data flow summary:

- Scheduler tick (every second) checks which configured sites are due

- Due site → HTTP(S) request + SSL cert inspection performed
- Result evaluated against rule set → state (UP/DOWN) + zero or more alerts produced
- Check result saved to SQLite; any alerts saved to SQLite and pushed to the live dashboard

4. Features Overview

4.1 Live Terminal Dashboard

A full-screen, auto-refreshing dashboard (4 updates/second) built with the Rich library, showing a per-site status table (state, HTTP code, latency, SSL expiry, uptime %), a status summary panel, and a recent-alerts feed.

4.2 Outage Detection

Connection failures, timeouts, and HTTP 5xx responses immediately mark a site DOWN and raise a CRITICAL alert. Recovery is automatically detected and logged as an INFO 'site back up' event.

4.3 Latency Monitoring

Every successful check records response time in milliseconds. Responses slower than a configurable threshold (default: 2000ms) raise a WARNING alert without marking the site down.

4.4 SSL Certificate Expiry Tracking

For HTTPS sites, PulseNex performs a live TLS handshake to read the certificate's expiry date. Certificates expiring within 14 days raise a WARNING; already-expired certificates raise a CRITICAL alert.

4.5 Content / Keyword Validation

An optional keyword can be configured per site. If the page returns HTTP 200 but the keyword is missing from the body, PulseNex raises a WARNING — catching cases like a broken deployment, maintenance page, or CMS error that still returns a 'successful' status code.

4.6 Persistent Local Storage & Reporting

All checks and alerts are stored in a local SQLite database. The 'report' command produces a plain-text uptime and incident summary for any time window, and the 'stats' command shows all-time alert breakdowns by severity, site, and category.

4.7 Demo Mode

A built-in simulated mode ('--demo' flag) drives the dashboard with synthetic sites and injected incidents (an outage, a slow response, an expiring certificate) so the tool can be demonstrated or screenshotted without needing real production traffic.

5. Technology Stack

Component	Technology
Language	Python 3.8+
HTTP Client	requests
Terminal UI	Rich (rich.live, rich.layout, rich.table)
SSL Inspection	Python 'ssl' + 'socket' standard library modules
Storage	SQLite 3 (built into Python standard library)
CLI	argparse (subcommands: add, remove, list, monitor, report, stats, clear)
Target OS	Linux (portable to any host with Python 3)

6. Alerting Rules

The following table lists every alert category produced by the evaluation engine in v1.0.0:

Category	Trigger	Severity	Notes
SITE_DOWN	Connection error, timeout, or HTTP 5xx	CRITICAL	Raised once on state change, not every failed check
SITE_RECOVERED	Site returns to a successful response after being down	INFO	Confirms restoration of service
SLOW_RESPONSE	Response time exceeds 2000ms (configurable)	WARNING	Site still counted as UP
KEYWORD_MISSING	Configured keyword not found in page body	WARNING	Catches silent content failures
SSL_EXPIRING	Certificate expires within 14 days (configurable)	WARNING	Gives time to renew before an outage
SSL_EXPIRED	Certificate expiry date has already passed	CRITICAL	Browsers will now block visitors

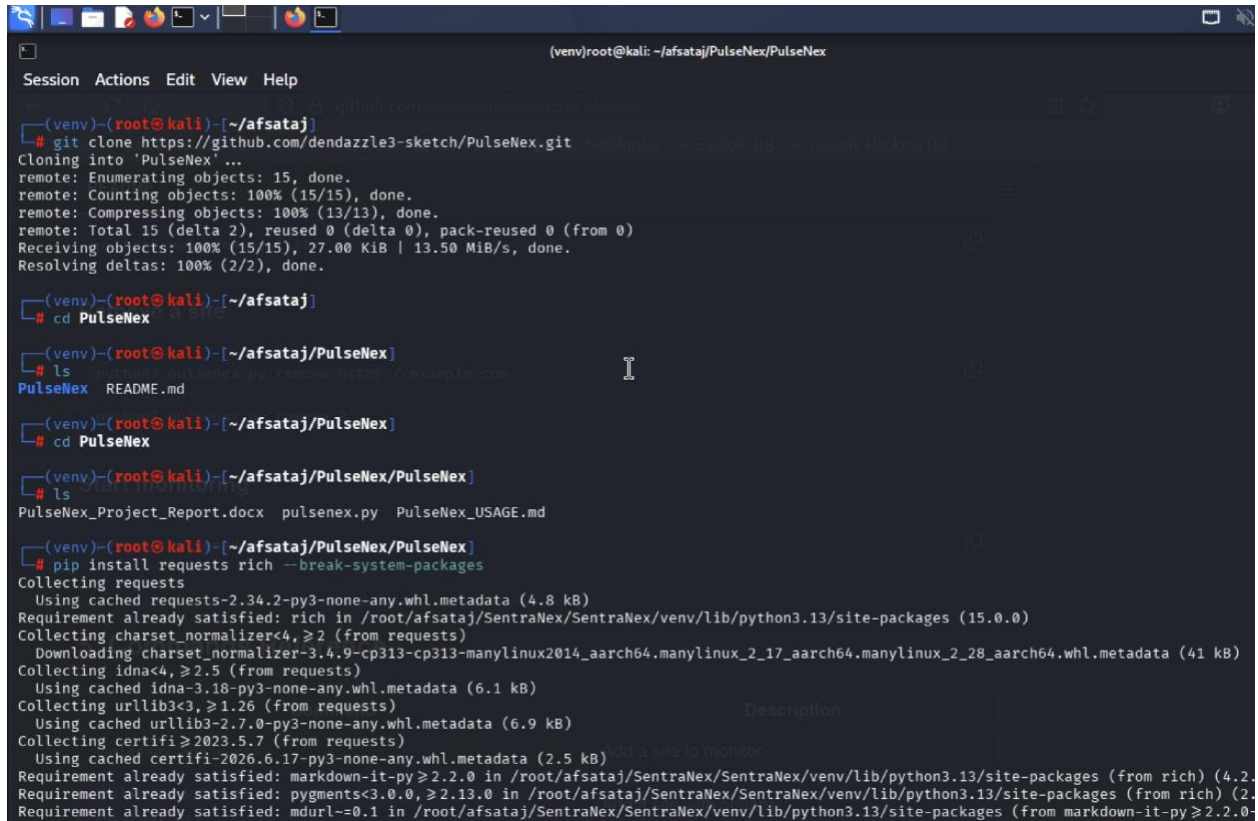
7. Installation & Deployment

Full setup and command reference is provided in the accompanying USAGE.md file. In summary:

- Requires Python 3.8+ and the 'requests' and 'rich' packages.
- Copy pulsenex.py to any Linux host (no compilation or build step needed).

- Try it instantly with simulated sites: `python3 pulsenex.py monitor --demo`
- Add real sites to monitor: `python3 pulsenex.py add https://example.com --name "My Site"`
- Start live monitoring: `python3 pulsenex.py monitor`
- All configuration and history persist automatically to `~/.pulsenex/pulsenex.db` between runs.

8. Screenshots & Demonstration



```
(venv)root@kali: ~/afsatay/PulseNex/PulseNex
Session Actions Edit View Help

(venv)~(root@kali)~[~/afsatay]
# git clone https://github.com/dendazze3-sketch/PulseNex.git
Cloning into 'PulseNex'...
remote: Enumerating objects: 15, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (13/13), done.
remote: Total 15 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (15/15), 27.00 KiB | 13.50 MiB/s, done.
Resolving deltas: 100% (2/2), done.

(venv)~(root@kali)~[~/afsatay]
# cd PulseNex

(venv)~(root@kali)~[~/afsatay/PulseNex]
# ls
PulseNex README.md

(venv)~(root@kali)~[~/afsatay/PulseNex]
# cd PulseNex

(venv)~(root@kali)~[~/afsatay/PulseNex/PulseNex]
# ls
PulseNex_Project_Report.docx pulsenex.py PulseNex_USAGE.md

(venv)~(root@kali)~[~/afsatay/PulseNex/PulseNex]
# pip install requests rich --break-system-packages
Collecting requests
  Using cached requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
Requirement already satisfied: rich in /root/afsatay/SentraNex/SentraNex/venv/lib/python3.13/site-packages (15.0.0)
Collecting charset_normalizer<4, >2 (from requests)
  Downloading charset_normalizer-3.4.9-cp313-cp313-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl.metadata (41 kB)
Collecting idna<4, >2.5 (from requests)
  Using cached idna-3.18-py3-none-any.whl.metadata (6.1 kB)
Collecting urllib3<3, >1.26 (from requests)
  Using cached urllib3-2.7.0-py3-none-any.whl.metadata (6.9 kB)
Collecting certifi<2026.6.17-py3-none-any.whl.metadata (2.5 kB)
Requirement already satisfied: markdown-it-py >=2.2.0 in /root/afsatay/SentraNex/SentraNex/venv/lib/python3.13/site-packages (from rich) (4.2)
Requirement already satisfied: pygments<3.0.0, >2.13.0 in /root/afsatay/SentraNex/SentraNex/venv/lib/python3.13/site-packages (from rich) (2.18.0)
Requirement already satisfied: mdurl->=0.1 in /root/afsatay/SentraNex/SentraNex/venv/lib/python3.13/site-packages (from markdown-it-py >=2.2.0-)
```

```

(venv)root@kali: ~/afsataj/PulseNex/PulseNex
Session Actions Edit View Help
Collecting requests
  Using cached requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
Requirement already satisfied: rich in /root/afsataj/SentraNex/SentraNex/venv/lib/python3.13/site-packages (15.0.0)
Collecting charset_normalizer<4, ≥2 (from requests)
  Downloading charset_normalizer-3.4.9-cp313-cp313-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl.metadata (41 kB)
Collecting idna<4, ≥2.5 (from requests)
  Using cached idna-3.18-py3-none-any.whl.metadata (6.1 kB)
Collecting urllib3<3, ≥1.26 (from requests)
  Using cached urllib3-2.7.0-py3-none-any.whl.metadata (6.9 kB)
Collecting certifi ≥2023.5.7 (from requests)
  Using cached certifi-2026.6.17-py3-none-any.whl.metadata (2.5 kB)
Requirement already satisfied: markdown-it-py ≥2.2.0 in /root/afsataj/SentraNex/SentraNex/venv/lib/python3.13/site-packages (from rich) (4.2.0)
Requirement already satisfied: pygments<3.0.0, ≥2.13.0 in /root/afsataj/SentraNex/SentraNex/venv/lib/python3.13/site-packages (from rich) (2.18.0)
Requirement already satisfied: mdurl=>0.1 in /root/afsataj/SentraNex/SentraNex/venv/lib/python3.13/site-packages (from markdown-it-py ≥2.2.0) (0.1.2)
Using cached requests-2.34.2-py3-none-any.whl (73 kB)
Downloading charset_normalizer-3.4.9-cp313-cp313-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl (214 kB)
Using cached idna-3.18-py3-none-any.whl (65 kB)
Using cached urllib3-2.7.0-py3-none-any.whl (131 kB)
Using cached certifi-2026.6.17-py3-none-any.whl (133 kB)
Installing collected packages: urllib3, idna, charset_normalizer, certifi, requests
Successfully installed certifi-2026.6.17 charset_normalizer-3.4.9 idna-3.18 requests-2.34.2 urllib3-2.7.0

(venv)-(root@kali)-[~/afsataj/PulseNex/PulseNex]
# python3 -m venv venv
source venv/bin/activate
pip install requests rich
Collecting requests
  Using cached requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
Collecting rich
  Using cached rich-15.0.0-py3-none-any.whl.metadata (18 kB)
Collecting charset_normalizer<4, ≥2 (from requests)
  Using cached charset_normalizer-3.4.9-cp313-cp313-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl.metadata (41 kB)
Collecting idna<4, ≥2.5 (from requests)
  Using cached idna-3.18-py3-none-any.whl.metadata (6.1 kB)
Collecting urllib3<3, ≥1.26 (from requests)
  Using cached urllib3-2.7.0-py3-none-any.whl.metadata (6.9 kB)
Collecting certifi ≥2023.5.7 (from requests)
  Using cached certifi-2026.6.17-py3-none-any.whl.metadata (2.5 kB)
Collecting markdown-it-py ≥2.2.0 (from rich)
  Using cached markdown_it_py-4.2.0-py3-none-any.whl.metadata (7.4 kB)
Collecting pygments<3.0.0, ≥2.13.0 (from rich)
  Using cached pygments-2.20.0-py3-none-any.whl.metadata (2.5 kB)
Collecting mdurl=>0.1 (from markdown-it-py ≥2.2.0->rich)
  Using cached mdurl-0.1.2-py3-none-any.whl.metadata (1.6 kB)
Using cached requests-2.34.2-py3-none-any.whl (73 kB)
Using cached charset_normalizer-3.4.9-cp313-cp313-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl (214 kB)
Using cached idna-3.18-py3-none-any.whl (65 kB)
Using cached urllib3-2.7.0-py3-none-any.whl (131 kB)
Using cached rich-15.0.0-py3-none-any.whl (310 kB)
Using cached pygments-2.20.0-py3-none-any.whl (1.2 MB)
Using cached certifi-2026.6.17-py3-none-any.whl (133 kB)
Using cached markdown_it_py-4.2.0-py3-none-any.whl (91 kB)
Using cached mdurl-0.1.2-py3-none-any.whl (10.0 kB)
Installing collected packages: urllib3, pygments, mdurl, idna, charset_normalizer, certifi, requests, markdown-it-py, rich
Successfully installed certifi-2026.6.17 charset_normalizer-3.4.9 idna-3.18 markdown-it-py-4.2.0 mdurl-0.1.2 pygments-2.20.0 requests-2.34.2
urllib3-2.7.0

```

```

(venv)root@kali: ~/afsataj/PulseNex/PulseNex
Session Actions Edit View Help

(venv)-(root@kali)-[~/afsataj/PulseNex/PulseNex]
# python3 -m venv venv
source venv/bin/activate
pip install requests rich
Collecting requests
  Using cached requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
Collecting rich
  Using cached rich-15.0.0-py3-none-any.whl.metadata (18 kB)
Collecting charset_normalizer<4, ≥2 (from requests)
  Using cached charset_normalizer-3.4.9-cp313-cp313-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl.metadata (41 kB)
Collecting idna<4, ≥2.5 (from requests)
  Using cached idna-3.18-py3-none-any.whl.metadata (6.1 kB)
Collecting urllib3<3, ≥1.26 (from requests)
  Using cached urllib3-2.7.0-py3-none-any.whl.metadata (6.9 kB)
Collecting certifi ≥2023.5.7 (from requests)
  Using cached certifi-2026.6.17-py3-none-any.whl.metadata (2.5 kB)
Collecting markdown-it-py ≥2.2.0 (from rich)
  Using cached markdown_it_py-4.2.0-py3-none-any.whl.metadata (7.4 kB)
Collecting pygments<3.0.0, ≥2.13.0 (from rich)
  Using cached pygments-2.20.0-py3-none-any.whl.metadata (2.5 kB)
Collecting mdurl=>0.1 (from markdown-it-py ≥2.2.0->rich)
  Using cached mdurl-0.1.2-py3-none-any.whl.metadata (1.6 kB)
Using cached requests-2.34.2-py3-none-any.whl (73 kB)
Using cached charset_normalizer-3.4.9-cp313-cp313-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl (214 kB)
Using cached idna-3.18-py3-none-any.whl (65 kB)
Using cached urllib3-2.7.0-py3-none-any.whl (131 kB)
Using cached rich-15.0.0-py3-none-any.whl (310 kB)
Using cached pygments-2.20.0-py3-none-any.whl (1.2 MB)
Using cached certifi-2026.6.17-py3-none-any.whl (133 kB)
Using cached markdown_it_py-4.2.0-py3-none-any.whl (91 kB)
Using cached mdurl-0.1.2-py3-none-any.whl (10.0 kB)
Installing collected packages: urllib3, pygments, mdurl, idna, charset_normalizer, certifi, requests, markdown-it-py, rich
Successfully installed certifi-2026.6.17 charset_normalizer-3.4.9 idna-3.18 markdown-it-py-4.2.0 mdurl-0.1.2 pygments-2.20.0 requests-2.34.2
urllib3-2.7.0

(venv)-(root@kali)-[~/afsataj/PulseNex/PulseNex]
# chmod +x pulsenex.py

(venv)-(root@kali)-[~/afsataj/PulseNex/PulseNex]
# python3 pulsenex.py monitor --demo

```


Session Actions Edit View Help

PulseNex Website Availability Monitor for Linux | uptime 0:01:20 | v1.0.0 | 4 site(s)

Monitored Sites

Site	State	HTTP	Latency	SSL Exp.	Uptime	Last Check
nexuswebsecurity.com	UP	200	211ms	162d	100.0%	18:28:30
client-portal.example.com	UP	200	294ms	95d	100.0%	18:28:30
shop.example.com	DOWN	-	-	26d	88.2%	18:28:30
api.example.com	UP	200	103ms	255d	88.2%	18:28:30

Optional: install as a system-wide command

sudo apt-get install pulse-nex

sudo systemctl enable pulse-nex

Status Summary

Total Sites
UP
DOWN
Pending

3. Quick Start (no configuration needed)

Recent Alerts

Time	Sev	Alert
18:28:30	CRITICAL	shop.example.com: shop.example.com is DOWN - Connection failed (host unreachable / DNS failure)
18:28:20	INFO	api.example.com: api.example.com is back UP (HTTP 200)
18:28:15	CRITICAL	api.example.com: api.example.com is DOWN - HTTP 503
18:28:15	WARNING	client-portal.example.com: client-portal.example.com responded slowly: 4181ms
18:27:50	INFO	shop.example.com: shop.example.com is back UP (HTTP 200)
18:27:45	CRITICAL	shop.example.com: shop.example.com is DOWN - Connection failed (host unreachable / DNS failure)
18:27:45	WARNING	client-portal.example.com: client-portal.example.com responded slowly: 2556ms
18:27:25	INFO	api.example.com: api.example.com is back UP (HTTP 200)
18:27:20	CRITICAL	api.example.com: api.example.com is DOWN - HTTP 503
18:27:15	WARNING	client-portal.example.com: client-portal.example.com responded slowly: 2951ms

Add a site

python3 pulse-nex.py add https://example.com

python3 pulse-nex.py add https://example.com

Session Actions Edit View Help

PulseNex Website Availability Monitor for Linux | uptime 0:00:00 | v1.0.0 | 2 site(s)

Monitored Sites

Site	State	HTTP	Latency	SSL Exp.	Uptime	Last Check
sbs.du.ac.in	UP	200	100ms	48d	100.0%	18:31:36
test	UNKNOWN	-	-	-	-	-

Remove a site

python3 pulse-nex.py remove https://example.com

python3 pulse-nex.py remove https://example.com

Status Summary

Total Sites
UP
DOWN
Pending

Start monitoring

Recent Alerts

Time	Sev	Alert
-	-	No alerts yet

5. Commands Reference

Command	Description
python3 pulse-nex.py add https://example.com	Add a site to monitor
python3 pulse-nex.py add https://example.com --interval 30s	Add a site with options
python3 pulse-nex.py remove https://example.com	Remove a monitored site

9. Testing & Validation

The tool was validated using the following approach before release:

- HTTP check logic tested against a live, reachable HTTPS site to confirm correct capture of status code, response time in milliseconds, and TLS certificate expiry.
- Failure handling tested against a non-resolving domain to confirm the DNS/connection-failure path is caught cleanly and produces a CRITICAL SITE_DOWN alert rather than a crash.
- State-change logic verified so that repeated DOWN checks do not spam duplicate alerts, and a SITE_RECOVERED alert fires exactly once when a site returns to UP.
- SSL expiry logic tested with both a healthy long-dated certificate and a simulated near-expiry certificate to confirm the WARNING threshold fires correctly.
- End-to-end test using the built-in --demo generator, including a simulated outage, a simulated slow response, and a simulated near-expiry certificate, to confirm the dashboard and alert feed render correctly together.
- 'report' and 'stats' commands verified against a populated test database to confirm correct filtering by time window and correct per-site uptime percentage calculation.

10. Future Enhancements

- Email / Telegram / Slack / Discord webhook notifications for CRITICAL alerts in real time.
- Multi-region checks (run checks from more than one vantage point to rule out local network issues).
- Public status-page generation (static HTML) from the stored uptime history.
- Configurable check methods beyond GET (HEAD requests, custom headers, POST with body, expected-status-code overrides).
- Historical uptime charts (daily/weekly/monthly) rendered in the terminal or exported to HTML.
- Distributed/agent mode for monitoring sites from multiple servers with a central aggregator.

11. Conclusion

PulseNex delivers the core functionality of a commercial uptime-monitoring service — outage detection, latency tracking, SSL expiry monitoring, and content validation — as a single, self-hosted Python file with no subscription cost and no third party holding the monitoring data. It is well suited for freelance client engagements where uptime reporting is part of a managed-hosting or website-maintenance package, as well as for personal VPS and side-project monitoring by Nexus Web Security.